Update Objects for Language Model Training: Replaying Training Progress Instead of Tokens

Julio Jose Lena
Independent Researcher
juliolena25@gmail.com
https://orcid.org/0009-0001-0817-6591

Abstract

Large language model training still treats text tokens as the main reusable unit of learning. If a training interval has already transformed parameters from θ_t to θ_{t+k}, then the interval also emits a causal object: the state transition that the optimizer actually bought. This paper introduces *update objects*, compact artifacts that replay useful training movement without replaying all source tokens. I study two concrete objects in small conventional decoder-only Transformers: sparse checkpoint delta replay objects and optimizer transaction capsules. All target arms start from the same θ_t and are compared against raw token replay, gradient-alignment selected replay, checkpoint distillation, no-source continuation, warm-start upper-bound controls, and shuffled-object ablations. Sparse checkpoint deltas are the strongest result: across two model widths, two text shards, compression sweeps, pessimistic build-cost accounting, and a five-seed d96 paper-core gate, they improve durable held-out cross-entropy progress per paid unit by 2.45–$2.90\times$ over the best non-object replay control. A more selective secondary object also survives: optimizer transaction capsules beat raw replay by $1.78\times$ in the same d96 five-seed paper-core gate, while mean-gradient, layerwise-gradient, alignment-weighted, and sign-consensus capsules fail or remain weak. These results do not claim production-scale speedup. They support a narrower thesis: previous training runs can produce reusable transition objects that are empirically different from data selection, distillation, checkpoint warm starts, or parameter-efficient adaptation.

1 Introduction

Modern language models are trained by repeatedly presenting text to an optimizer, measuring loss, and changing parameters. Once the step is complete, the tokens, gradients, optimizer state, and parameter movement are usually discarded or reduced to logs. Later work that needs the same capability must typically replay the text, select a smaller subset of text, distill from a teacher, warm-start from a checkpoint, or fine-tune a low-rank adapter. Those methods reuse examples, functions, or weights. They rarely treat the training transition itself as reusable data.

This paper asks a different question: after a corpus block has already moved a model from θ_t to θ_{t+k}, can the useful part of that movement be stored as a compact object and replayed into another compatible training state? The motivating unit is not a token and not a teacher logit. It is an update object: an artifact whose payload is a priced, causal summary of what the training interval did to the model.

The distinction is important. Data selection methods choose which examples to train on; update objects attempt to preserve the effect of a completed training interval. Dataset distillation synthesizes examples that induce useful future updates; update objects store the update-side artifact directly. Function preserving growth and model merging reuse trained functions or parameter states; update

Preprint.

objects are evaluated as replayable training progress under an explicit cost denominator. This boundary makes the proposal testable: an update object should buy more durable held-out cross-entropy reduction per paid unit than replaying the original tokens or using nearby reuse baselines.

The practical motivation is not that token replay is obsolete. Tokens remain the ground truth interface for language. The claim is narrower: modern training runs already produce expensive intermediate evidence about how a model changes when a block of data is consumed. If that evidence can be stored as a compact transition object, then failed runs, pilot runs, refresh runs, and corpus passes may become a source of reusable training data rather than only sunk cost.

I make four contributions.

1. I define *update objects* as priced, replayable summaries of a training-state transition, and separate them from examples, logits, adapters, checkpoint warm starts, and ordinary model diffs.
2. I introduce two concrete objects for conventional Transformer gates: sparse checkpoint delta replay objects and optimizer transaction capsules.
3. I define a durable cross-entropy progress score that charges object build, application, replay, selection, distillation, and continuation costs in token-backward-equivalent units.
4. I report falsifier-style gates in small decoder-only Transformers, showing that checkpoint deltas are consistently strong and that only the optimizer-transaction version of capsules survives mature controls.

The results are intentionally modest in scale but strict in control design. Every arm begins from the same θ_t. Object arms are compared to raw token replay, gradient-alignment selected replay, checkpoint distillation, no-source continuation, warm-start upper bounds, and shuffled-object ablations. The most important run is a d96, five-seed paper-core gate: checkpoint delta objects score 3.624947 durable CE progress per paid unit, optimizer transaction capsules score 2.231399, and raw replay scores 1.250833. The corresponding ratios are $2.898\times$ and $1.784\times$ against raw replay.

2 Related Work

Data selection and influence. LESS selects influential instruction-tuning examples by building reusable low-rank gradient features and ranking examples by similarity to a target capability [Xia et al., 2024]. This paper uses a similar intuition that gradient-side information is valuable, but the object boundary is different: the selected unit is not an example. The replayed unit is a parameter-space transition object emitted by a source interval.

Dataset distillation and trajectory matching. Dataset distillation compresses a dataset into synthetic examples that train a model efficiently [Wang et al., 2018]. Trajectory matching pushes distilled data so that synthetic training follows expert training trajectories [Cazenavette et al., 2022]. Update objects move one step closer to the trajectory itself. Rather than synthesizing inputs that induce a trajectory, they store sparse pieces of the trajectory or optimizer transaction and test whether those pieces can be replayed directly.

Function-preserving growth and parameter reuse. Net2Net transfers a function into wider or deeper networks [Chen et al., 2015], and LiGO learns growth operators for pretrained Transformers [Wang et al., 2023]. LoRA stores task adaptation in low-rank parameter updates [Hu et al., 2022]. Task arithmetic defines task vectors as differences between pretrained and fine-tuned weights and uses arithmetic in weight space to edit model behavior [Ilharco et al., 2023]. TIES-Merging trims small deltas, resolves sign conflicts, and merges only parameters aligned with an elected sign [Yadav et al., 2023]. DARE drops and rescales delta parameters before merging homologous language models [Yu et al., 2024]. These methods are the closest prior-art pressure for the checkpoint-delta result. Update objects are not claimed to replace them, and the paper should not be read as saying that sparse deltas are new by themselves. The distinction is the experimental contract: the object is scored as replayable paid training progress from a source interval, against token replay, selected replay, distillation, continuation, warm-start collapse, and shuffled-object controls.

3 Update Objects

3.1 Training Intervals

Let θ_t be a model state and let $B_{t:t+k-1}$ be a sequence of source minibatches. A training rule U maps

$$\theta_{i+1} = U(\theta_i, B_i) \tag{1}$$

for $i = t, \ldots, t+k-1$, producing a terminal state θ_{t+k}. The usual record of this interval is the source data and perhaps a checkpoint. An update object treats the interval as an object-producing computation:

$$O_{t,k} = E(\theta_t, \theta_{t+k}, B_{t:t+k-1}, U, M), \tag{2}$$

where E is an encoder and M denotes optional measured side information such as gradients, optimizer steps, or coordinate-wise movement. The object is useful only if a replay operator R can apply it to a compatible state:

$$\widehat{\theta}_{t+k} = R(\theta_t, O_{t,k}). \tag{3}$$

This paper does not require exact reconstruction of θ_{t+k}. The target is cheaper durable progress on held-out cross-entropy after replay and a short continuation interval.

3.2 Durable Progress Per Paid Unit

Let D_{eval} be a held-out evaluation split and let $\mathrm{CE}(\theta; D_{\mathrm{eval}})$ be mean next-token cross-entropy. The initial held-out loss is

$$L_0 = \mathrm{CE}(\theta_t; D_{\mathrm{eval}}). \tag{4}$$

For any arm a, the gate first applies the arm-specific replay operation, then trains for the same continuation budget, producing θ_a^+. The durable progress is

$$\Delta_{\mathrm{dur}}(a) = L_0 - \mathrm{CE}(\theta_a^+; D_{\mathrm{eval}}). \tag{5}$$

The paid score divides this durable progress by an accounting denominator:

$$S(a) = \frac{\Delta_{\mathrm{dur}}(a)}{C_{\mathrm{replay}}(a) + C_{\mathrm{build}}(a) + C_{\mathrm{apply}}(a) + C_{\mathrm{select}}(a) + C_{\mathrm{distill}}(a)}. \tag{6}$$

The costs in this paper are token-backward-equivalent proxy units. They are not claimed to be hardware FLOPs. The purpose is to prevent an object from winning by hiding its preprocessing or teacher cost. A later systems paper would need wall-clock or profiler-derived FLOP accounting; the present paper only tests whether the object mechanism survives fair first pricing.

3.3 Admissible Controls

The control set is part of the definition. An update object is not meaningful if it only beats doing nothing. It should beat or clearly complement:

- **raw replay**, which trains on the original source batches;
- **selected replay**, which pays for gradient-alignment selection and trains on the selected source batches;
- **distillation**, which trains from θ_t against θ_{t+k} logits on the same source batches;
- **no-source continuation**, which spends only the continuation budget without source replay;
- **warm-start checkpoint**, an upper-bound collapse control that starts from θ_{t+k} but is charged a larger source-build cost;
- **shuffled ablations**, which preserve object density and magnitude distribution while destroying coordinate identity.

These controls make the paper falsifiable. A method that loses to raw replay is not a useful replay object. A method that loses to its shuffled ablation is not using the claimed coordinate information. A method that only wins by ignoring build or selection cost is not an update-object gain.

4 Two Concrete Update Objects

4.1 Sparse Checkpoint Delta Replay Objects

The first object is the simplest transition summary:

$$\Delta_{t,k} = \theta_{t+k} - \theta_t. \tag{7}$$

A full checkpoint delta is too close to warm-starting the target from a source checkpoint, so the gate uses a sparse transition object. Let $\mathrm{TopK}_\rho(v)$ keep the largest ρ fraction of coordinates by absolute value and set all other coordinates to zero. The checkpoint delta object is

$$O^{\Delta}_{t,k} = \mathrm{TopK}_\rho(\Delta_{t,k}), \qquad R_\Delta(\theta_t, O^{\Delta}_{t,k}) = \theta_t + \alpha_\Delta O^{\Delta}_{t,k}. \tag{8}$$

In the gates below, $\alpha_\Delta = 1$. The object is therefore a sparse, coordinate-preserving replay of the state movement bought by the source interval. Its shuffled ablation keeps the same values and density but permutes coordinates. This is the key sanity check: if the object only works because it adds noise or changes norm, the shuffled version should also work.

This object is intentionally vulnerable to prior-art pressure. It resembles checkpoint compression, model-diff transfer, task arithmetic, DARE-style delta sparsification, and sparse model merging. The paper's escape boundary is not "a sparse delta exists." The boundary is that the delta is scored as a replayable training-transition object under durable CE progress per paid unit, against token replay and distillation controls.

4.2 Optimizer Transaction Capsules

The second object records optimizer-side movement rather than the endpoint difference. Let the source interval produce step transactions

$$u_i = \theta_{i+1} - \theta_i. \tag{9}$$

The signed transaction over the interval is

$$T_{t,k} = \sum_{i=t}^{t+k-1} u_i, \tag{10}$$

and the coordinate activity score is

$$A_{t,k}[j] = \sum_{i=t}^{t+k-1} |u_i[j]|. \tag{11}$$

The optimizer transaction capsule keeps the signed transaction values on the coordinates with largest cumulative movement:

$$O^{\mathrm{opt}}_{t,k}[j] = \begin{cases} T_{t,k}[j] & \text{if } j \in \mathrm{TopK}_\rho(A_{t,k}), \\ 0 & \text{otherwise.} \end{cases} \tag{12}$$

The replay operator is direct addition,

$$R_{\mathrm{opt}}(\theta_t, O^{\mathrm{opt}}_{t,k}) = \theta_t + \alpha_{\mathrm{opt}} O^{\mathrm{opt}}_{t,k}, \tag{13}$$

with $\alpha_{\mathrm{opt}} = 1$ in the reported gates.

The activity score matters. A mean gradient capsule stores a sparse average gradient and applies a single learning-rate-scaled update. That object was too weak in the gates. The surviving capsule is different: it stores the sparse sum of actual optimizer transactions and selects coordinates by cumulative movement. In other words, the object preserves what the optimizer repeatedly used during the source interval, not merely what the average gradient suggested at the beginning.

4.3 Why the Objects Are Not Equivalent

For plain gradient descent with fixed learning rate, no momentum, no adaptive state, and no nonlinearity from changing batches, the sum of optimizer transactions can collapse toward a checkpoint delta. In realistic training, the objects separate. The checkpoint delta is an endpoint summary. The optimizer capsule is a transaction summary with its own activity criterion. The empirical gates reflect this separation: checkpoint deltas are stronger, but optimizer transaction capsules survive where gradient-only capsules do not.

5 Experimental Protocol

5.1 Models and Data

The experiments use small conventional decoder-only Transformers implemented in PyTorch. The gates are deliberately not ternary-SSM gates; the purpose is to test whether the update-object idea has a conventional-language-model form. I use model widths d48, d64, and d96, AdamW optimization, byte-token training shards derived from Wikipedia-like text, and held-out next-token cross-entropy. The reported tables use two shard sizes: a smaller `wiki_1mb` shard for early falsifiers and a larger `wiki_5mb` shard for the more stable gates.

Each run has the same high-level structure. First, a source interval trains a model from θ_t to θ_{t+k} and emits candidate update objects. Second, every replay arm is reset to the same θ_t. Third, each arm applies its replay operation or source replay. Fourth, each arm receives the same continuation budget. The final score is computed from held-out CE after this continuation phase. This design avoids the most common false positive: giving an object arm a hidden warm start or a different continuation budget.

5.2 Arms

The core arms are:

- **raw replay**: replay the source batches with ordinary AdamW;
- **selected replay**: replay a selected subset of source batches chosen by gradient alignment, with selection cost charged;
- **checkpoint distillation**: train from θ_t against θ_{t+k} teacher logits on the same source batches;
- **no-source continuation**: spend only the continuation budget;
- **checkpoint delta object**: apply Eq. 8;
- **optimizer transaction capsule**: apply Eq. 12;
- **gradient capsule variants**: mean-gradient, layerwise-gradient, alignment-weighted, and sign-consensus capsules;
- **shuffled ablations**: density- and magnitude-matched shuffled versions of the object arms;
- **warm-start checkpoint**: initialize from θ_{t+k} and charge a larger source-build cost.

Warm-start is not treated as a deployable object in the main claim. It is a collapse control: if an object merely approximates starting from the terminal checkpoint, then warm-start should dominate when the source-build cost is not counted. The stricter question is whether a compact object can beat replay after its own construction and application costs are charged.

5.3 Cost Accounting

The raw replay denominator is the number of source replay token-backward units plus the shared continuation units. Object arms pay a build cost, an application cost, and the same continuation cost. Selection and distillation arms pay their own selection or teacher costs. Several gates deliberately vary the checkpoint-delta build cost to test whether the result survives pessimistic accounting. The resulting score is therefore a *paid-unit* gate, not a claim about optimized wall-clock training time.

This distinction is central to the paper. A production system could implement objects inefficiently and lose wall-clock time; conversely, a kernel-level implementation might make object application cheaper than the proxy used here. The empirical claim is only that, under explicit first pricing, some transition objects buy more durable held-out CE progress than replaying the source tokens.

6 Sparse Checkpoint Delta Results

Table 1 reports the main checkpoint-delta gates. The object wins in every listed condition. The first three runs use three seeds. The paper-lite follow-up uses five seeds on `wiki_5mb` with d64, and the checkpoint delta object wins all five seeds against the best non-object replay/control.

Table 1: Sparse checkpoint delta replay objects. Scores are durable held-out CE progress per paid unit.

Run	Data	d	Tokens	Ratio	Delta	Raw	Distill
d48 main	`wiki_1mb`	48	60k	2.487	0.600784	0.241524	0.199657
d64 main	`wiki_1mb`	64	80k	2.456	1.206092	0.491047	0.385313
d64 larger shard	`wiki_5mb`	64	150k	2.454	1.215255	0.495229	0.390310
d64 paper-lite	`wiki_5mb`	64	180k	2.468	1.165835	0.472301	0.368169

Table 2: Checkpoint-delta sensitivity gates.

Gate	Change	Object/control ratio
Compression 0.005	stronger sparsity	1.871
Compression 0.05	more transition mass	2.638
Build cost 10	pessimistic object build cost	1.379

The result is not just a win over an unusually weak baseline. In the same runs, gradient-alignment selected replay is close to raw replay but does not beat the checkpoint delta object, and checkpoint distillation is weaker still. The shuffled-delta ablation is also lower than the true coordinate-preserving delta. In the paper-lite run, for example, the shuffled-delta score is 0.400597 while the true delta score is 1.165835. This gap matters because it shows that coordinate identity is doing work; the object is not only adding a sparse perturbation with the right magnitude distribution.

6.1 Compression and Build-Cost Sensitivity

Table 2 tests whether the checkpoint delta object survives stronger compression or pessimistic build accounting. At density 0.005, the ratio falls but remains positive at $1.871\times$. At density 0.05, the ratio rises to $2.638\times$. When the object build cost is raised to ten paid units, the ratio drops to $1.379\times$ but remains above raw replay. This is important for paper honesty: the object is not free, and a sufficiently expensive implementation could erase the gain. The current claim is that the mechanism survives first-order pricing pressure.

The stronger d96 paper-core gate appears in the combined results below because it also includes the mature capsule family. Its checkpoint-delta result is the strongest single measurement in the paper: 3.624947 paid progress versus 1.250833 for raw replay, a $2.898\times$ ratio over the best non-object control.

7 Optimizer Transaction Capsule Results

The initial capsule result was weak because the first capsule was effectively a sparse average-gradient object. That object did not beat raw replay. The mature gate therefore separates gradient-only capsules from optimizer transaction capsules. Table 3 summarizes the surviving object and the failure modes.

The capsule is therefore not as strong as the checkpoint delta object, but it is not dead. It has a clear operating region. It needs enough source interval length, enough density, and direct access to optimizer transactions. It fails when over-compressed to density 0.005. It remains positive under a pessimistic build-cost setting but drops below a strict $1.20\times$ paper threshold. The d96 paper-core gate is the cleanest result: the capsule scores 2.231399, raw replay scores 1.250833, and the shuffled optimizer capsule scores 0.987417.

7.1 Gradient-Only Capsules Are Not Enough

The mature gate also tested mean-gradient, layerwise-gradient, alignment-weighted, and sign-consensus capsules. These are useful negative controls. In the d96 paper-core gate, their shuffled and unshuffled scores sit near the same low band, around 0.987 paid progress for the shuffled variants. They do not establish a viable update object. This narrows the paper's claim: "capsules work" would

Table 3: Mature optimizer transaction capsule gates. Scores are durable held-out CE progress per paid unit.

Run	Model	Density	Capsule	Raw	Shuffled	Ratio
source24	d64	0.02	0.506960	0.472301	0.380432	1.073
source48	d64	0.02	0.635740	0.489593	0.398688	1.298
source48	d64	0.05	0.904204	0.489593	0.399149	1.847
source48	d64	0.005	0.483575	0.489593	0.398317	0.988
build-cost 10	d64	0.05	0.561091	0.489593	0.247686	1.146
d96 scale check	d96	0.05	2.267360	1.261991	0.971832	1.797
d96 paper-core	d96	0.05	2.231399	1.250833	0.987417	1.784

Table 4: d96 five-seed paper-core gate.

Arm	Paid progress score	Ratio to raw replay
Checkpoint delta object	3.624947	2.898
Optimizer transaction capsule	2.231399	1.784
Raw replay	1.250833	1.000
Shuffled delta ablation	1.039598	0.831
Shuffled optimizer capsule	0.987417	0.789

be too broad, but "optimizer transaction capsules can survive when the object records actual optimizer movement" is supported by the gates.

This distinction is mathematically plausible. A gradient-only object records a local direction estimate. An optimizer transaction object records the realized path after AdamW scaling, batch order, adaptive state, and repeated coordinate use. If the goal is to replay a training interval rather than predict a single step, the realized transaction is the more faithful object.

8 Combined Paper-Core Gate

The final d96 paper-core gate uses five seeds, source/replay lengths $48/12$, and density 0.05. It is the most important table because it tests the strongest checkpoint delta and the mature capsule in the same conventional Transformer setting.

The combined gate supports two conclusions. First, sparse checkpoint deltas are the main paper result. They beat raw replay, distillation, selected replay in earlier gates, and shuffled coordinate controls. Second, optimizer transaction capsules are a secondary but meaningful result. They are weaker than checkpoint deltas, but they beat raw replay and their shuffled capsule control in all five paper-core seeds.

9 Discussion

9.1 Why This Is More Than Checkpoint Compression

The strongest objection is that a sparse checkpoint delta is merely checkpoint compression or task-vector reuse. That objection is partially correct and must not be hidden. The object payload is a sparse parameter delta, and adjacent literature already treats weight-space deltas as editable, mergeable, and sparsifiable objects. The new part is the experimental contract around that payload. It is not evaluated as a storage artifact, a model merge, a task edit, or a final checkpoint. It is evaluated as a replayable training-transition object that competes with token replay, selected replay, distillation, continuation, warm-start collapse controls, and shuffled coordinate ablations under a paid progress denominator.

This reframing changes the practical question. A lab already stores many checkpoints and logs from pilot runs, failed runs, curriculum sweeps, data refreshes, and ablations. If those runs can emit reusable transition objects, then training history becomes a cache of paid movement. The paper-core

result does not prove that this cache scales to frontier LLMs, but it makes the question concrete enough to test.

9.2 Why Capsules Still Matter

Checkpoint deltas are strong but endpoint-centered. Optimizer transaction capsules point at a different family of objects: path-centered training artifacts. A capsule can, in principle, store where the optimizer repeatedly spent movement even when the endpoint delta hides cancellation, adaptive rescaling, or path structure. The mature gates show that a naive gradient capsule is not enough, but an optimizer transaction capsule can survive. That is a smaller claim than the initial capsule hope, but it is also cleaner.

The next capsule direction should not be another mean-gradient variant. It should preserve more optimizer semantics: moments, update signs over time, block-local activity, curvature tags, or compatibility conditions between the source and replay states. The paper therefore treats capsules as a promising secondary line rather than the headline result.

10 Limitations

The evidence has five important limitations.

First, the experiments are small. The models are conventional decoder-only Transformers, but they are not production-scale LLMs. The result should be read as a falsifier-passing mechanism, not as proof of frontier-scale training acceleration.

Second, the cost denominator is a proxy. Paid units charge source replay, build, selection, distillation, application, and continuation in token-backward-equivalent units. They are useful for avoiding free-object accounting, but they are not profiler-measured FLOPs, memory traffic, or wall-clock time.

Third, the strongest object is vulnerable to prior-art pressure. Sparse checkpoint deltas intersect with checkpoint compression, task arithmetic, model-diff transfer, DARE/TIES-style delta processing, and model merging. The paper's novelty depends on the update-object contract: durable replay progress per paid unit against token replay and distillation controls. A larger paper should include stronger same-budget baselines: magnitude-pruned task vectors, DARE/TIES-like delta processing, low-rank or SVD deltas, quantized sparse deltas, and same-storage-budget checkpoint compression.

Fourth, compatibility is only lightly tested. All target arms begin from the same θ_t used by the source interval. A future system must define how far a replay state can drift before an object becomes unsafe or useless. The next decisive gate should apply the same object to nearby checkpoints, another seed, or a controlled $\theta_t + \epsilon$ drift state.

Fifth, quality is measured by held-out cross-entropy only. Capability-level effects, long-horizon behavior, instruction-following quality, and safety behavior are outside the current gate.

11 Conclusion

This paper introduced update objects: compact, priced, replayable summaries of training-state transitions. In small conventional decoder-only Transformers, sparse checkpoint delta replay objects consistently beat raw token replay, gradient-alignment selected replay, checkpoint distillation, continuation controls, and shuffled coordinate ablations in durable held-out CE progress per paid unit. The strongest d96 five-seed gate gives a $2.898\times$ ratio over raw replay. Optimizer transaction capsules are weaker but survive as a secondary path-centered object, reaching $1.784\times$ over raw replay in the same gate, while gradient-only capsules remain weak.

The central claim is therefore not that token replay is obsolete, nor that a production LLM training run will immediately accelerate by the reported ratios. The central result is that training intervals can emit reusable transition artifacts whose value is separable from token replay, distillation, and shuffled perturbations under explicit paid-progress accounting. If this boundary scales, language model training could treat previous runs as sources of replayable training progress rather than only as sources of final checkpoints and logs.

References

George Cazenavette, Tongzhou Wang, Antonio Torralba, Alexei A. Efros, and Jun-Yan Zhu. Dataset distillation by matching training trajectories. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2Net: Accelerating learning via knowledge transfer. *arXiv preprint arXiv:1511.05641*, 2015.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations*, 2023.

Peihao Wang, Rameswar Panda, Lucas Torroba Hennigen, Philip Greengard, Leonid Karlinsky, Rogerio Feris, David Daniel Cox, Zhangyang Wang, and Yoon Kim. Learning to grow pretrained models for efficient transformer training. In *International Conference on Learning Representations*, 2023.

Tongzhou Wang, Jun-Yan Zhu, Antonio Torralba, and Alexei A. Efros. Dataset distillation. *arXiv preprint arXiv:1811.10959*, 2018.

Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. LESS: Selecting influential data for targeted instruction tuning. In *International Conference on Machine Learning*, 2024.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems*, 2023.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *International Conference on Machine Learning*, 2024.